

Spring JDBC

For building any application which will connect to the db and can perform the operations on the database we need APIs

Select, update, delete, insert
[Create table] ————— [Material data table/db]
[dummy db]

JDBC APIs

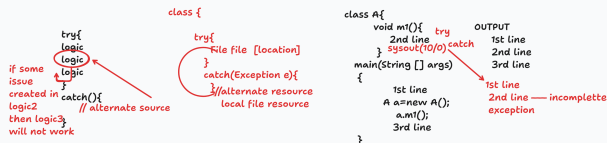
There are several drawbacks present inside the JDBC API

1. if we are using the jdbc in our application for connecting with the db in our application we have to write without any reason a set of try and catch block which have no use

JDBC api provide some set of classes through which we can connect to the database

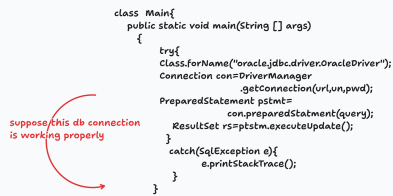
Connection
Statement/Prepared statement
Result set

These all throw a checked exception and if as a programmer if we are using any class and that class functionality is throwing checked exception then must and should we have to write try and catch block otherwise we will get compile time error



checked Exception is known as recoverable exception because must and should we have to write try and catch and meaning of writing try and catch is handling the exception as well as providing the alternate resources

Unchecked exception is known as non recoverable because without try and catch block the functionality will work without compile time error and if any exception occur or coming from method we don't have anything to catch or provide alternate resource



2. Resource Management

suppose we are writing the set of code for connecting with db then we should have to open the resource and we have to close the resource in the same order if we missed the order in future the possibility of causing some issue

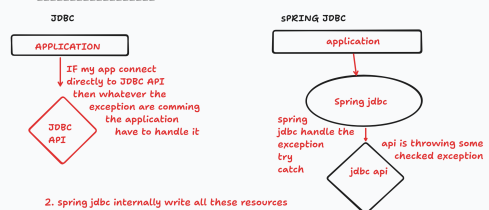
```

class Main {
    public static void main(String [] args) {
        try {
            Class.forName("oracle.jdbc.driver.OracleDriver");
            Connection con=DriverManager.getConnection(url,un,pwd);
            PreparedStatement pstmt=con.prepareStatement(query);
            ResultSet rs=pstmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
  
```

rs.close();
pstmt.close();
con.close();

3. Boiler plate logic

Spring JDBC



2. spring jdbc internally write all these resources that's why the closing resources is the responsibility of spring jdbc

For Connecting with the DB

```

1. Load Driver ----- Class.forName()
2. Create Connection ----- Connection con=DriverManager.getConnection(url,un,pwd);
3. Create statement ----- PreparedStatement pstmt=con.prepareStatement
4. Writing the Query ----- Select * from Table where uid=?
5. set the PARAMETER ----- pstmt.setInt(1, id);
6. Resultset ----- ResultSet rs=pstmt.executeUpdate();
7. Iterating over the database ----- while(rs!=null)
8. closing the resource ----- rs.close(), pstmt.close(), con.close()
  
```

Spring JDBC

ARTH number
1-3 and 5-8 all these Spring jdbc internally manage

HOW ?

For sending any sql query and taking the back the result in JDBC API

Statement
PreparedStatement
JdbcTemplate
NameParameterJdbcTemplate